



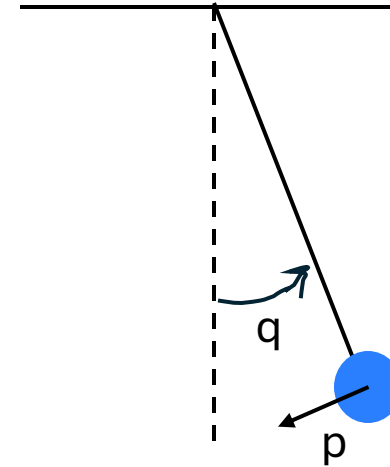
Hybrid modelling for the ocean using deep learning

Maya Janvier, Julie Deshayes, Julien
Salomon, Etienne Meunier



Summary

- I. Context and motivation
- II. A first ODE: damped pendulum
 - Implementation of APHYNITY
 - Results
 - Simulation
 - Interpretability
 - Generalization
- III. Numerical and analytical properties
 - Influence of time scheme
 - Lipschitz constants
 - A first upper bound on trajectory error
- IV. Towards ocean modelling
 - Py-qg





I. Context: ocean modeling

Numerical model

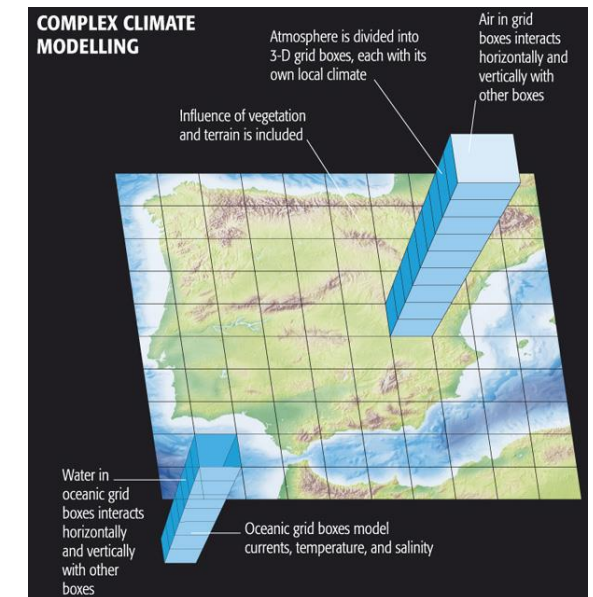
$$\frac{\partial X_t}{\partial t} = f(X_t)$$

+ Solver (time and space)

Challenges:

- Gridded Earth: missing HR dynamics → parametrisations

$$\frac{\partial X_t^{LR}}{\partial t} = f(X_t^{LR}) + R_\theta(X_t^{LR})$$



→ How can we learn a corrective R_θ to get HR-like dynamics within LR system ?



I. Context: ocean modeling

- **What is R_θ ?**
 - Neural Network (NN)

$$R_\theta = NN_\theta = (\sigma_N \circ f_N) \circ (\sigma_{N-1} \circ f_{N-1}) \circ \dots \circ (\sigma_0 \circ f_0)$$

with $f_i(x) = W_i^T x + \beta_i$ linear layer

σ_i non-linear activation functions (ex: ReLU)

- Reduced order model

$$R_\theta = \sum_{i=1}^N \theta_i g_i$$

Ex: $g_i = \Delta$



I. Context: deep learning for improving simulations

Numerical model

$$\frac{\partial X_t}{\partial t} = f(X_t)$$

NEMO

Neural ODE

$$\frac{d\tilde{X}_t}{dt} = f_\theta(\tilde{X}_t)$$

Chen et al. (2019)

+ Solver (time and space)

Imbalanced settings:

- f , describing known physical dynamics, can be incomplete and miss some physics that are embedded in observed data
- f_θ , although relying on observed data, is not constrained to produce trajectories that are physically correct + **unstable training**



I. Context: Neural ODE limitations

$$\frac{d\tilde{X}_t}{dt} = f_\theta(\tilde{X}_t) \text{ to model } \frac{dX_t}{dt} = f(X_t)$$

- Trajectory loss:

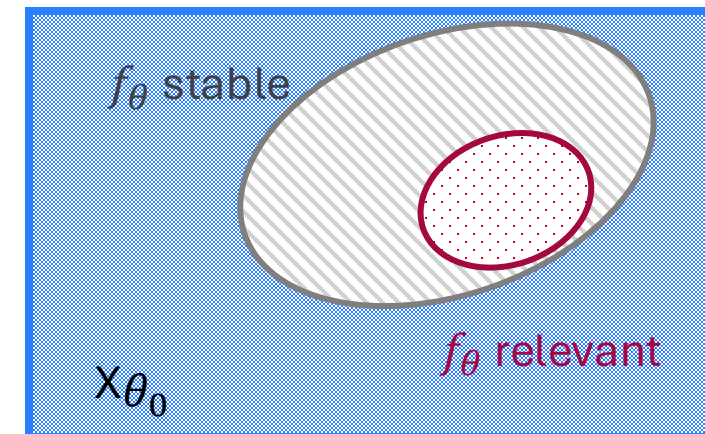
$$\hat{\theta} = \operatorname{argmin}_{\theta} \mathcal{L}(\theta) = \operatorname{argmin}_{\theta} \sum_{h=1}^{T/\Delta t} \left\| X_{h\Delta t}^{(i)} - \tilde{X}_{h\Delta t}^{\theta, (i)} \right\|$$

- Training:

$$\theta_0 \sim \mathcal{N}(0, 1)$$

$$\theta_{n+1} = \theta_n - \alpha \nabla_{\theta} \mathcal{L}(\theta_n)$$

In practice: unstable during training, not SOTA in terms of performances (Frezat 2023)



x d dimensions ...



I. Motivation: benefit from both worlds

Numerical model

$$\frac{\partial \tilde{X}_t}{\partial t} = f(\tilde{X}_t)$$

NEMO

Augmented dynamics

$$\frac{\partial \tilde{X}_t}{\partial t} = f(\tilde{X}_t) + f_\theta(\tilde{X}_t)$$

Yin, Frezat

Neural ODE

$$\frac{d\tilde{X}_t}{dt} = f_\theta(\tilde{X}_t)$$

Chen et al. (2019)

+ Solver (time and space)

$$f_\theta = NN_\theta = (\sigma_N \circ f_N) \circ (\sigma_{N-1} \circ f_{N-1}) \circ \dots \circ (\sigma_0 \circ f_0)$$

with $f_i(x) = W_i^T x + \beta_i$ and σ_i non-linear activation functions (ex: ReLU)



I. Hybrid modelling

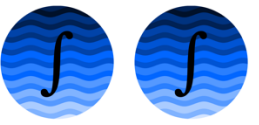
Augmented dynamics

$$\frac{\partial \tilde{X}_t}{\partial t} = f(\tilde{X}_t) + f_\theta(\tilde{X}_t) \quad \text{Yin, Frezat}$$

Motivations:

- Use the prior physical knowledge that we have: less hard task to complete an imperfect dynamic than simulating it entirely
- Could also correct solver numerical mistakes

→ How to train a hybrid model with augmented dynamics and a solver ?



II. A first ODE: damped pendulum

APHYNITY, Yin et al. 2021

- **Training dataset:**

$$X_t = \begin{pmatrix} q(t) \\ p(t) \end{pmatrix} \quad F^{(\omega_0, \alpha)}(X_t) = \begin{pmatrix} \dot{q}(t) \\ \dot{p}(t) \end{pmatrix} = \begin{pmatrix} p(t) \\ -\omega_0^2 \sin(q(t)) - \alpha p(t) \end{pmatrix}, \quad \omega_0 = \frac{2\pi}{12}, \alpha = 0.2$$

- **Learn to simulate trajectories with deep learning**

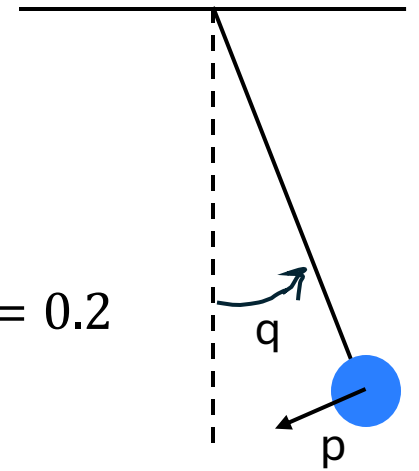
$$\frac{d\tilde{X}_t}{dt} = F_p^{\theta_p}(\tilde{X}_t) + F_a^{\theta_a}(\tilde{X}_t), \quad \Theta = (\theta_p, \theta_a)$$

Baseline: Neural ODE NN_{θ_a}

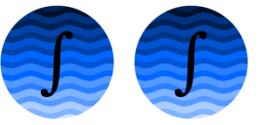
$$\frac{d\tilde{X}_t}{dt} = F_a^{\theta_a}(\tilde{X}_t) = NN_{\theta_a}(\tilde{X}_t)$$

Augmented dynamics: ParamODE(ω_0) + NN_{θ_a}

$$\frac{d\tilde{X}_t}{dt} = F_p^{(\omega_0)}(\tilde{X}_t) + F_a^{\theta_a}(\tilde{X}_t) = \begin{pmatrix} \tilde{p}(t) \\ -\omega_0^2 \sin(\tilde{q}(t)) \end{pmatrix} + NN_{\theta_a}(\tilde{X}_t)$$



X : true trajectory
 $\tilde{X}$: predicted trajectory

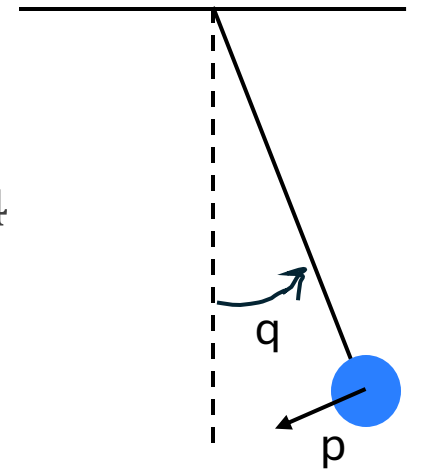


II. A first ODE: training setup

APHYNITY, Yin et al. 2021

$\{X_{h\Delta t}^{(i)}, 0 \leq h \leq \lfloor \frac{T}{\Delta t} \rfloor\}_{1 \leq i \leq n}$ generated damped trajectories, $T = 20, \Delta t = 0.5, RK4$

$\{\tilde{X}_{h\Delta t}^{(\theta_p, \theta_a)(i)}, 0 \leq h \leq \lfloor \frac{T}{\Delta t} \rfloor\}_{1 \leq i \leq n}$, i-th learned trajectory, $T = 20, \Delta t = 0.5, RK4$



- Training models ie **fitting θ_p, θ_a** : 2 losses

→ trajectory loss

$$\mathcal{L}_{traj}(\theta_p, \theta_a) = \sum_{i=0}^N \sum_{h=1}^{T/\Delta t} \|X_{h\Delta t}^{(i)} - \tilde{X}_{h\Delta t}^{(\theta_p, \theta_a)(i)}\|$$

→ trajectory loss + penalisation (APHYNITY)

$$\begin{aligned} \mathcal{L}_{\lambda_j}(\theta_p, \theta_a) &= \|F_a^{\theta_a}\| + \lambda_j \mathcal{L}_{traj}(\theta_p, \theta_a) \\ \lambda_{j+1} &= \lambda_j + \tau_2 \mathcal{L}_{traj}(\theta_{j+1}) \end{aligned}$$

With **gradient descent (Adam)**, jax

Algorithm 1: APHYNITY

Initialization:

$\lambda_0 \geq 0, \tau_1 > 0, \tau_2 > 0;$

for $epoch = 1 : N_{epochs}$ **do**

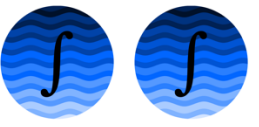
for $iter$ in $1 : N_{iter}$ **do**

for $batch$ in $1 : B$ **do**

$\theta_{j+1} = \theta_j -$

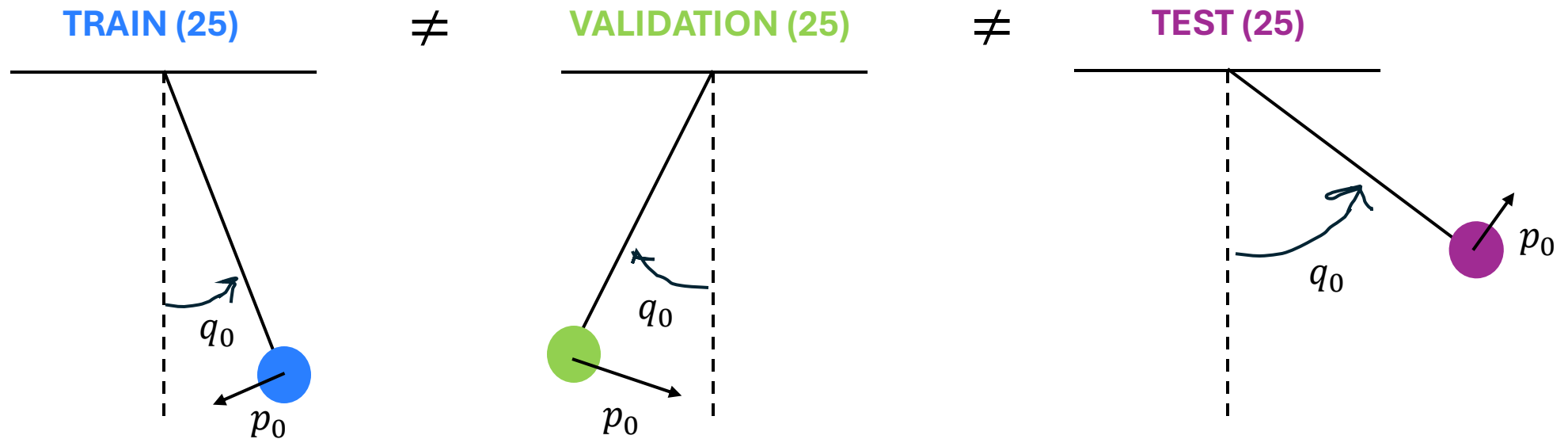
$\tau_1 \nabla [\lambda_j \mathcal{L}_{traj}(\theta_j) + \|F_a\|]$

$\lambda_{j+1} = \lambda_j + \tau_2 \mathcal{L}_{traj}(\theta_{j+1})$



II. A first ODE: training setup

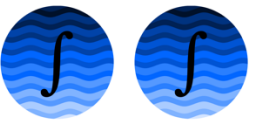
APHYNITY, Yin et al. 2021



- Minimize loss on train set to optimize (θ_p, θ_a)

- Avoid overfitting (learning to predict the train set)
- Select best model

- Evaluate performances on unseen situations

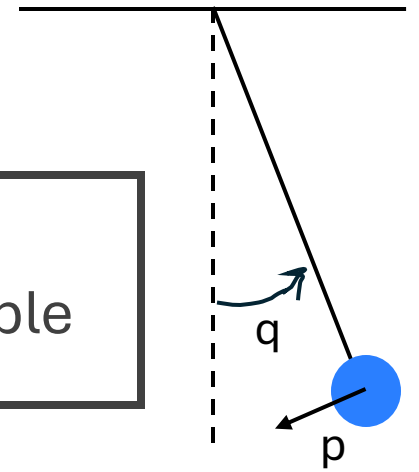


II. A first ODE: methodology

APHYNITY, Yin et al. 2021

Problem

Learn to complete missing physics with deep learning in toy example



Q1. Simulation:

Given (q_0, p_0) , is the model able to simulate the true damped trajectories ?

Q2. Interpretability: For $\text{ParamODE}(\omega_0) + NN_\theta$,

2.1 Do we recover the true ω_0 when fitting the augmented model?

2.2 Is the neural network able to simulate $(q, p) \rightarrow (0, -\alpha p)$?

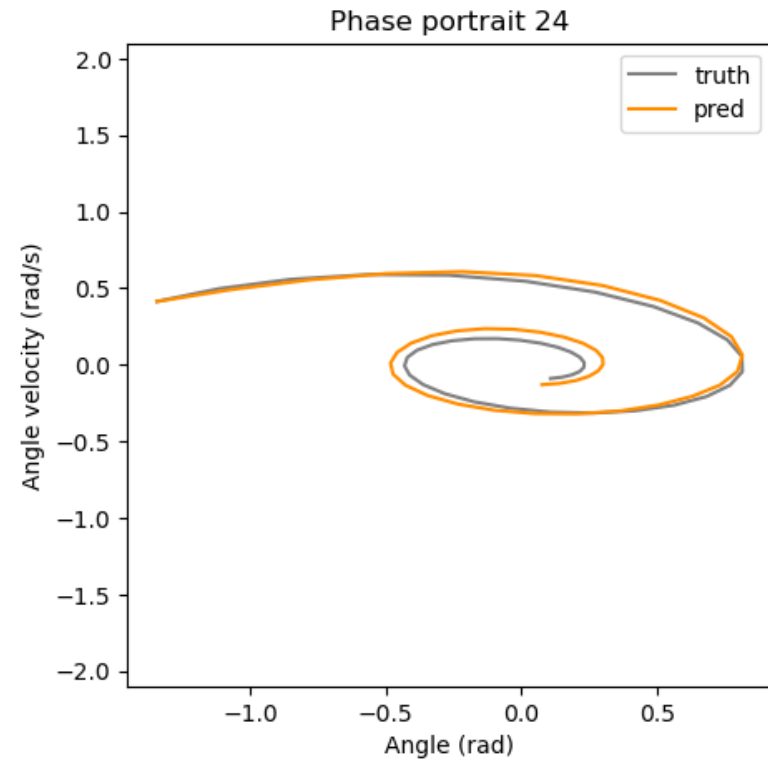
Q3. Generalization:

3.1 Is the learned hybrid dynamic able to generalize well to unseen states ?

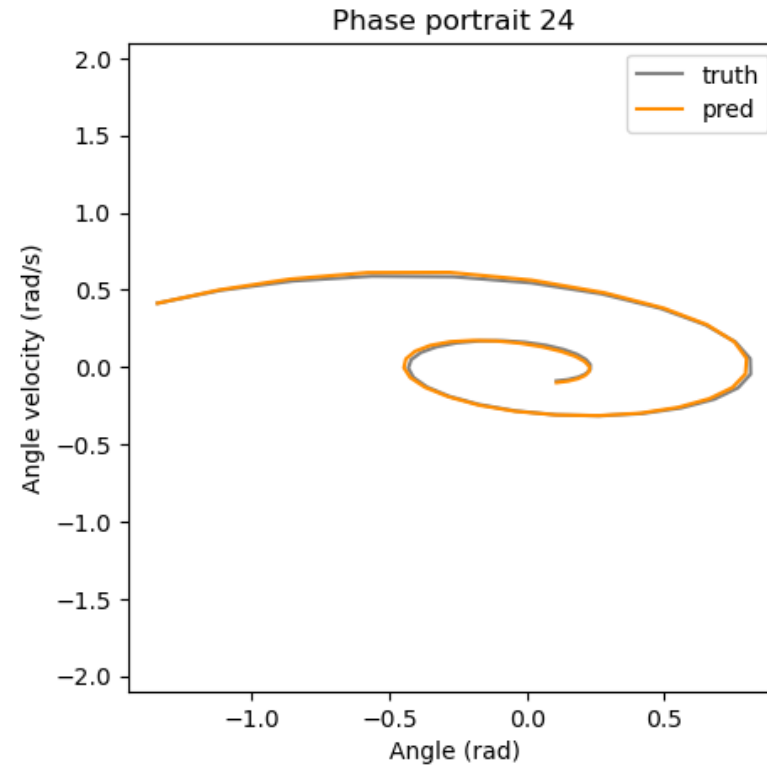
3.2 Stationarity and robustness of model trajectories for out-of-distribution initial conditions

Q1. Simulation: Given (q_0, p_0) , is the model able to simulate the correct trajectories ?

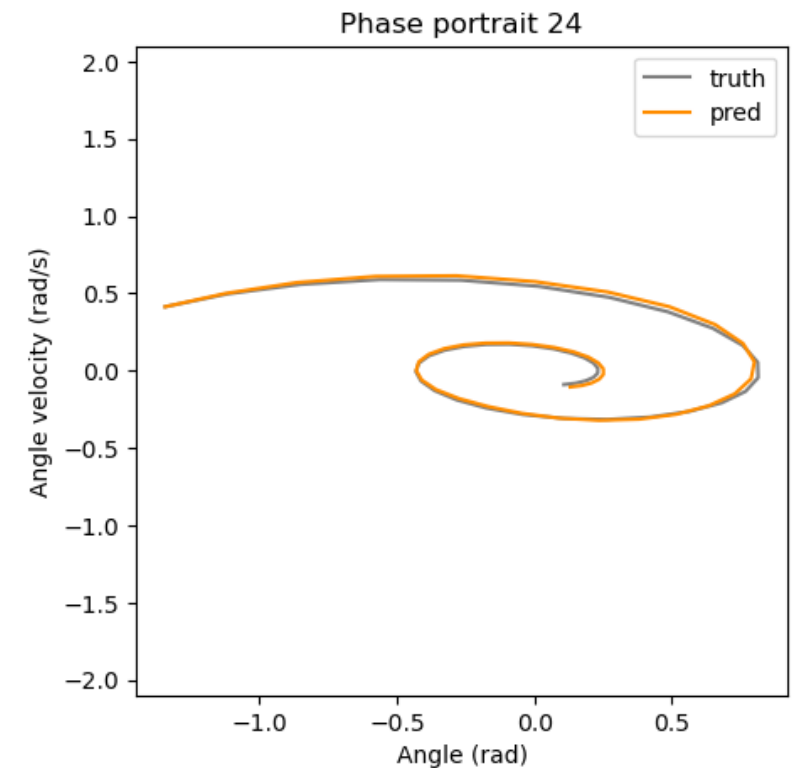
NN_{θ_a} – trajectory loss



ParamODE(ω_0) + NN_{θ_a} – trajectory loss

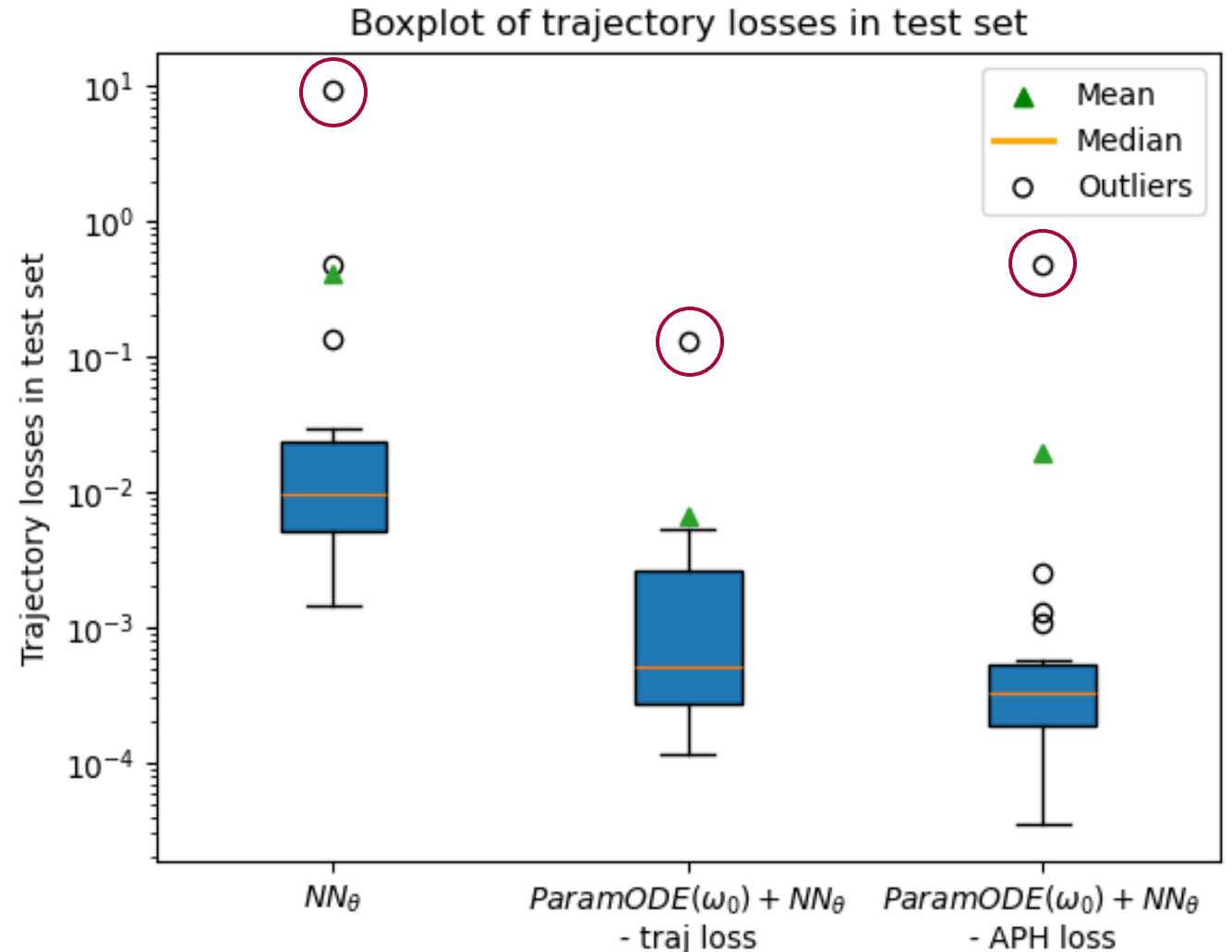


ParamODE(ω_0) + NN_{θ_a} – APHYNITY loss



Q1. Simulation: Given (q_0, p_0) , is the model able to simulate the correct trajectories ?

- NN_{θ_a} worst performances and less robust
- **ParamODE(ω_0) + NN_{θ_a} – trajectory loss** predicts good trajectories but less precise than with APHYNITY loss
- **ParamODE(ω_0) + NN_{θ_a} – APHYNITY loss** gets smallest losses but also an undesirable outlier (trajectory where they all struggle).



Q2.1 Interpretability: Impact of the penalisation on the learned $\hat{\omega}_0$

True $\omega_0^2 = 0.2742$

$\hat{\omega}_0^2 = 0.2103$

$\hat{\omega}_0^2 = 0.2608$

- Why?

$$F^{(\hat{\omega}_0, \theta_a)} \begin{pmatrix} q \\ p \end{pmatrix} = \begin{pmatrix} p \\ -\hat{\omega}_0^2 \sin q \end{pmatrix} + \begin{pmatrix} F_a^{\theta_a}(q, p)[0] \\ F_a^{\theta_a}(q, p)[1] \end{pmatrix}$$

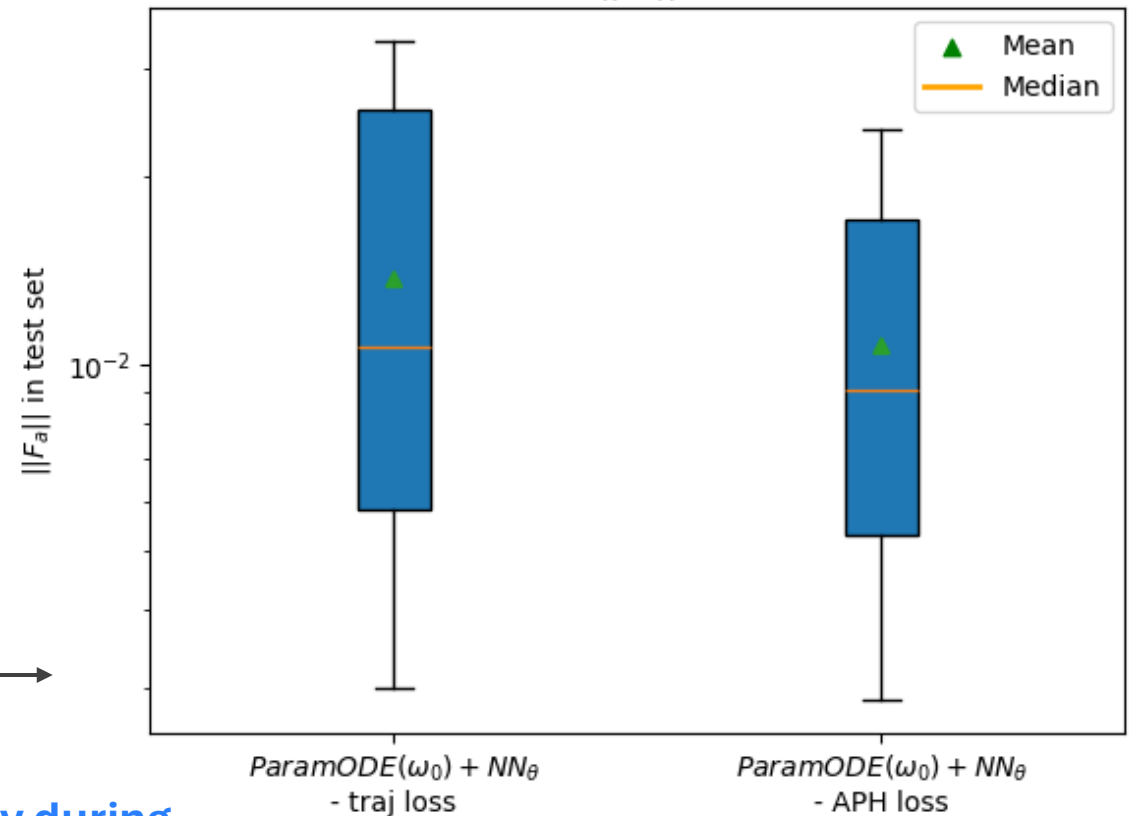
$$= \begin{pmatrix} p \\ -\omega_0^2 \sin q - \alpha p \end{pmatrix} + \underbrace{\begin{pmatrix} F_a^{\theta_a}(q, p)[0] \\ -(\hat{\omega}_0^2 - \omega_0^2) \sin q + \alpha p + F_a^{\theta_a}(q, p)[1] \end{pmatrix}}_{\text{What loss should enforce to 0}}$$

$\Rightarrow F_a^{\theta_a}[1]$ has to compensate more the further $\hat{\omega}_0^2$ is from the true parameter



$$\mathcal{L}_{traj/APH}(\hat{\omega}_0, \theta_a) = \sum_{i=0}^N \sum_{h=1}^{T/\Delta t} \left\| X_{h\Delta t}^{(i)} - \tilde{X}_{h\Delta t}^{(\hat{\omega}_0, \theta_a)(i)} \right\| + \left\| F_a^{\theta_a} \right\|$$

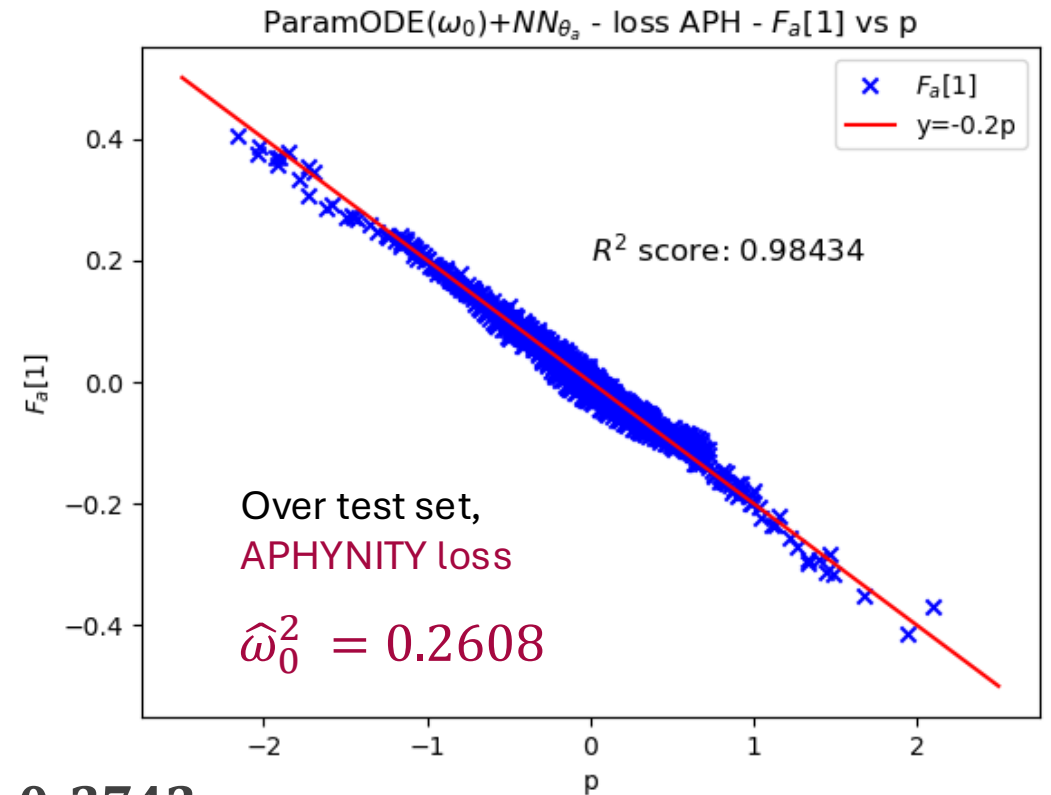
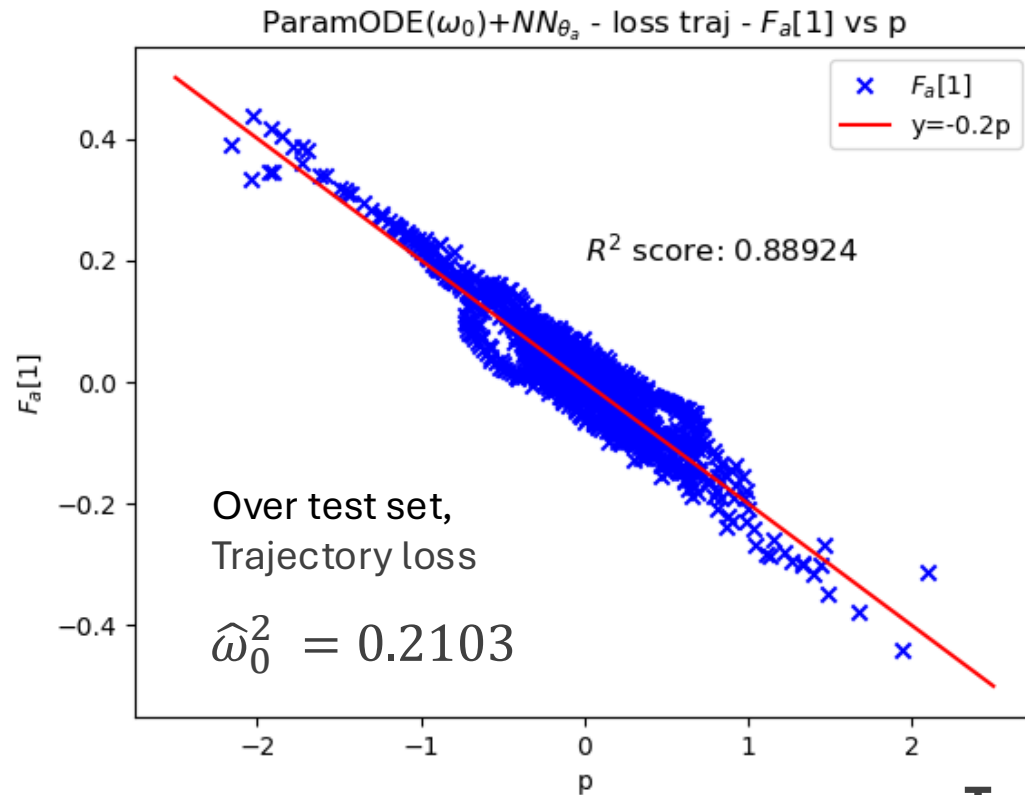
Boxplot of $\|F_a\|$ in test set



→ Adding a penalisation on $F_a^{\theta_a}$ minimizes this quantity during training to allow a better learning of the parameters

Q2.2 Interpretability: Is the neural network able to simulate $(q, p) \rightarrow (0, -\alpha p)$?

$$F_p^{(\hat{\omega}_0)}(\tilde{X}_t) = \begin{pmatrix} \tilde{p}(t) \\ -\hat{\omega}_0^2 \sin(\tilde{q}(t)) \end{pmatrix} \quad F_a^{\theta_a}(\tilde{X}_t) = NN_{\theta_a}(\tilde{X}_t) \text{ so we want } F_a^{\theta_a} \begin{pmatrix} \tilde{q} \\ \tilde{p} \end{pmatrix} = \begin{pmatrix} 0 \\ -\alpha \tilde{p} \end{pmatrix}$$

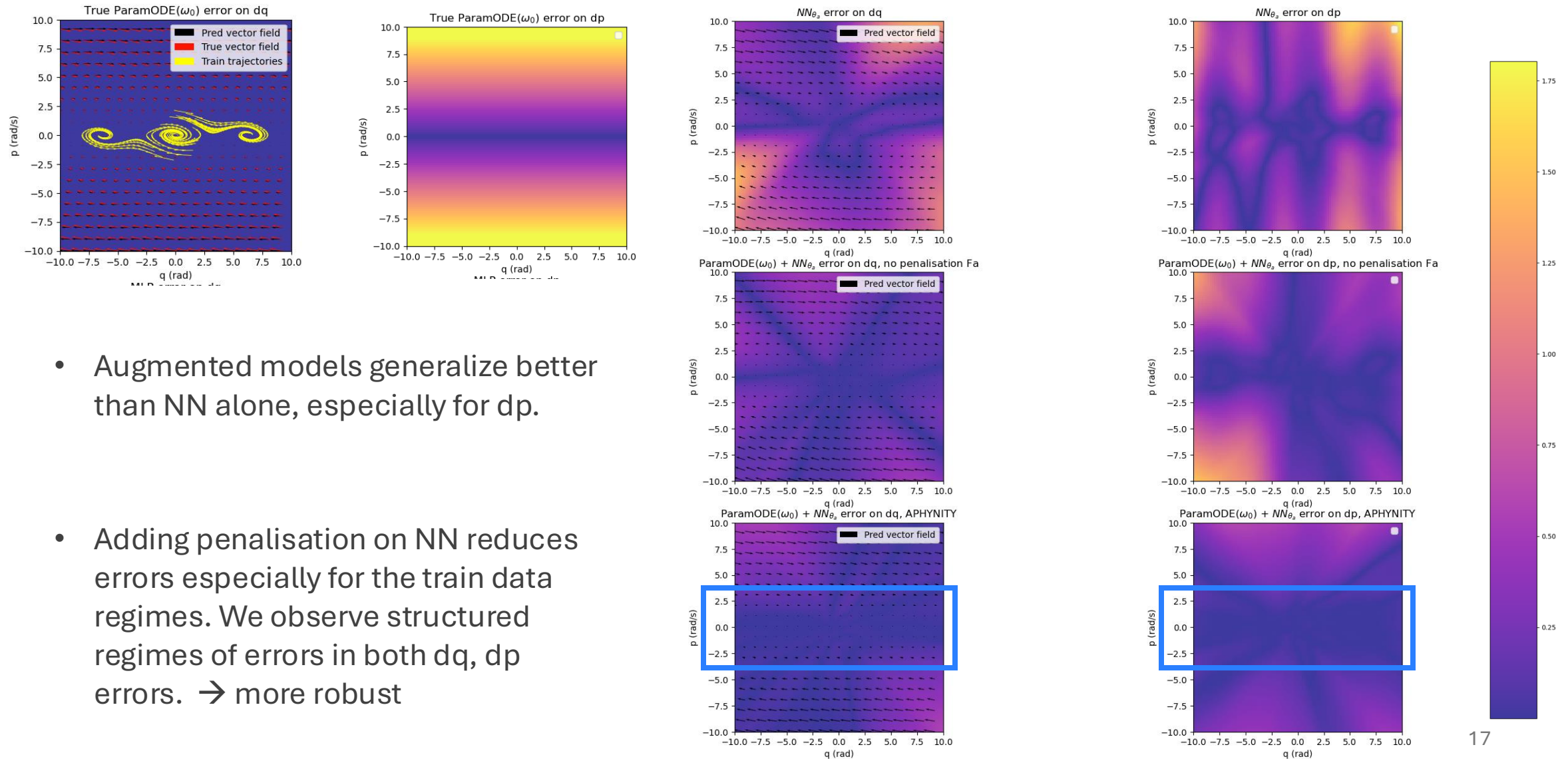


True $\omega_0^2 = 0.2742$

The better the ω_0 , the better the α . Without the constraint the learned ω_0 is further from the true one, which impacts the quality of $F_a^{\theta_a}$ and the « effective » α . **This penalization thus enhances interpretability.**

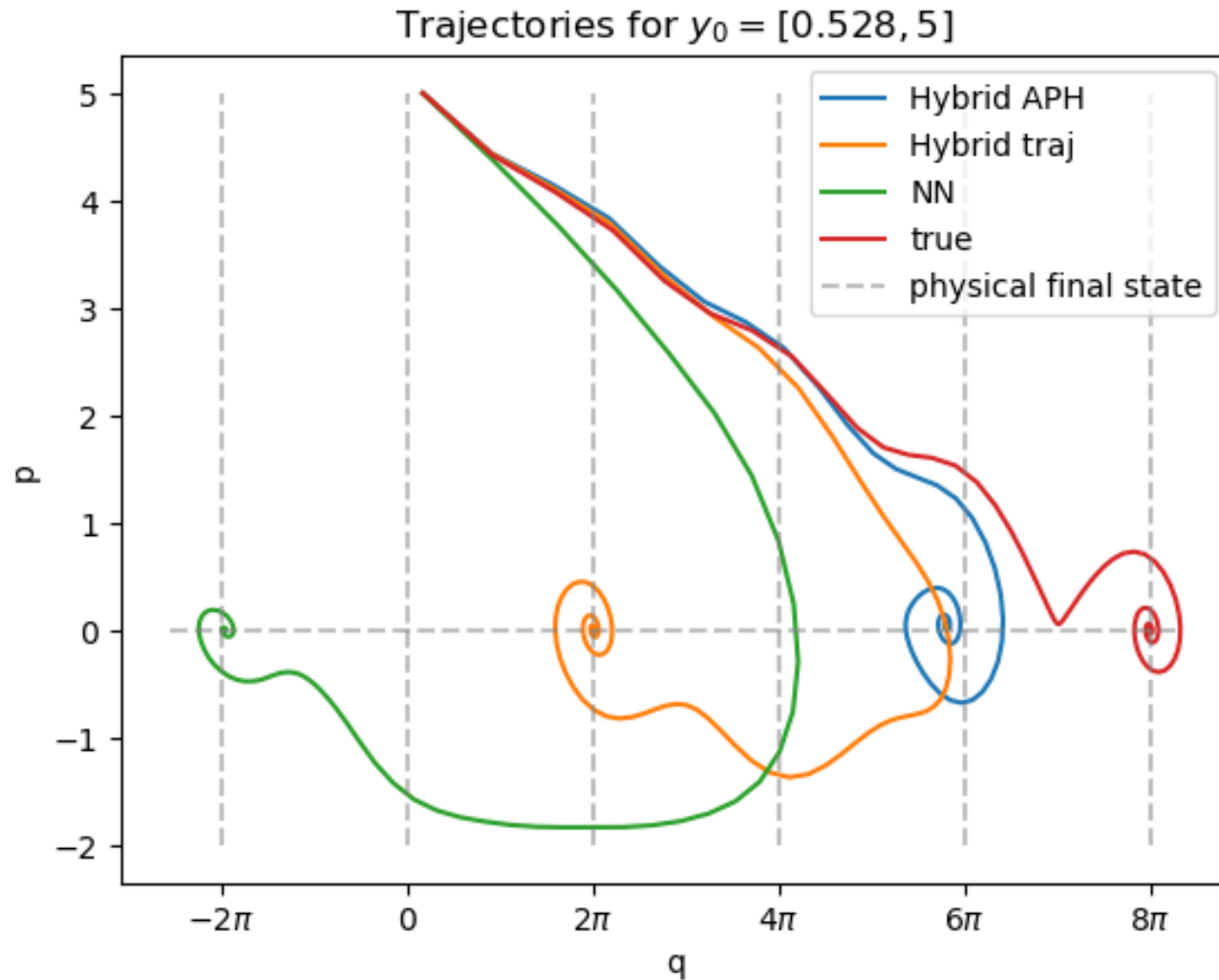
Q3.1 Generalization: Is the learned hybrid dynamic able to generalize well to unseen states ?

Absolute errors on dq and dp between true and model vector fields



- Augmented models generalize better than NN alone, especially for dp.
- Adding penalisation on NN reduces errors especially for the train data regimes. We observe structured regimes of errors in both dq, dp errors. → more robust

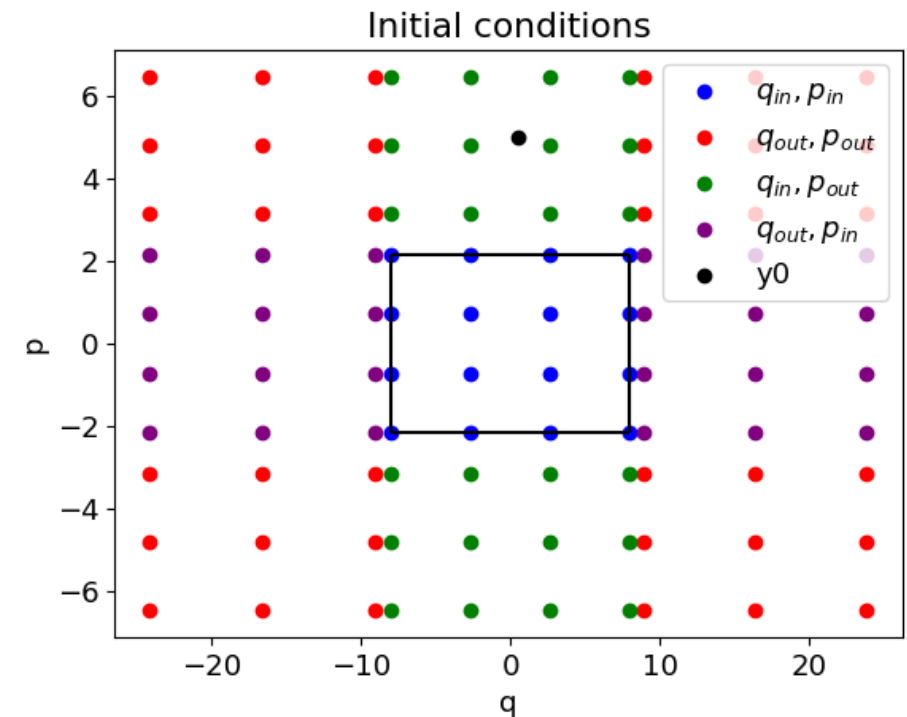
Q3.2 Generalization: Stationarity and robustness of trajectories outside training domain



→ All trajectories are stationary !

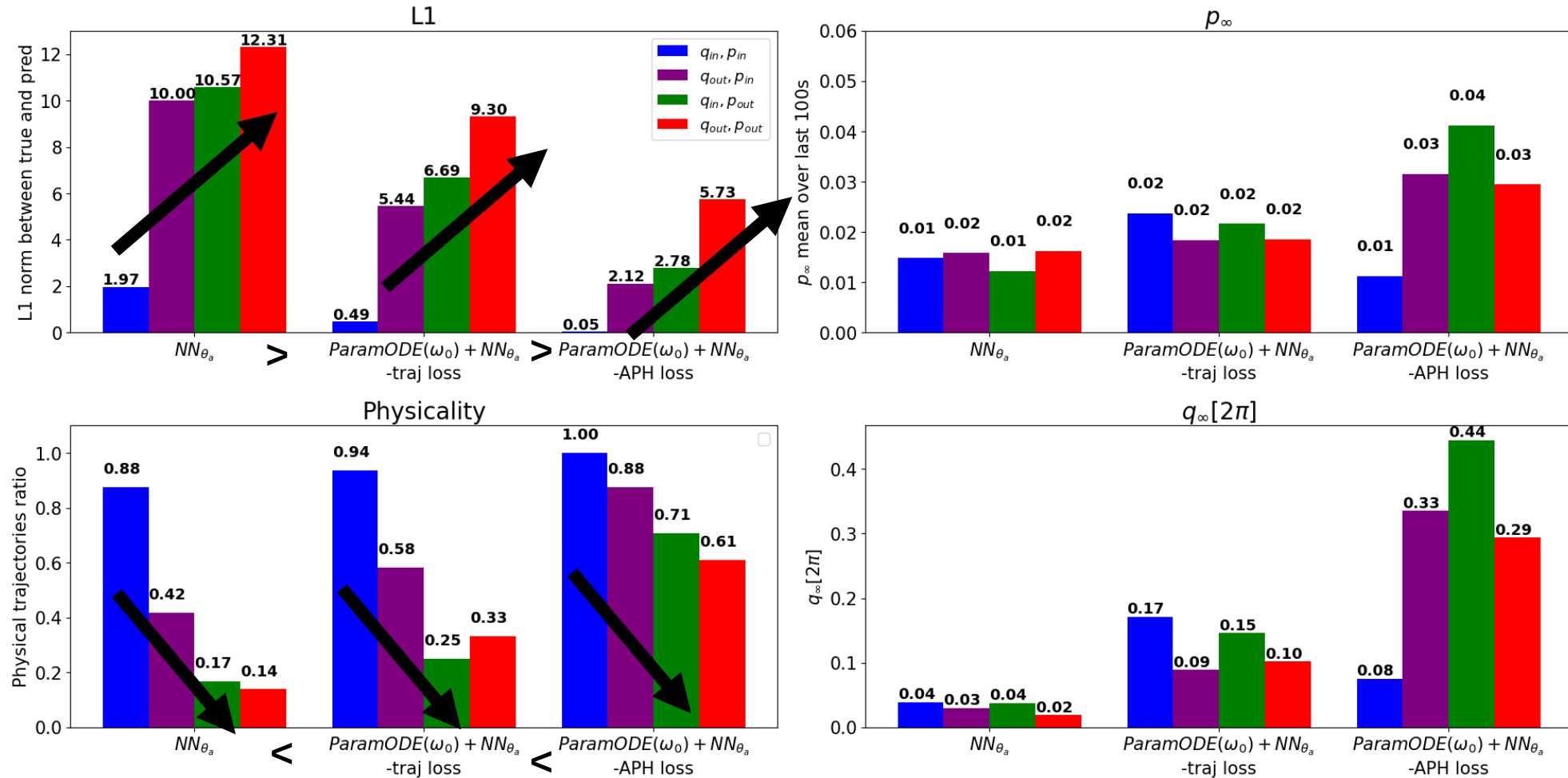
Tests at infinity (400s):

- Stationary ?
- Performance L1 traj ?
- Final $p = 0$? (true: $1e-7$)
- Final $q = 0[2\pi]$? (true: $1e-7$)
- Physical trajectories: some do turns in one direction and then the other



Q3.2 Generalization: Stationarity and robustness for out-of-distribution initial conditions

Comparison of metrics for different models and initial conditions

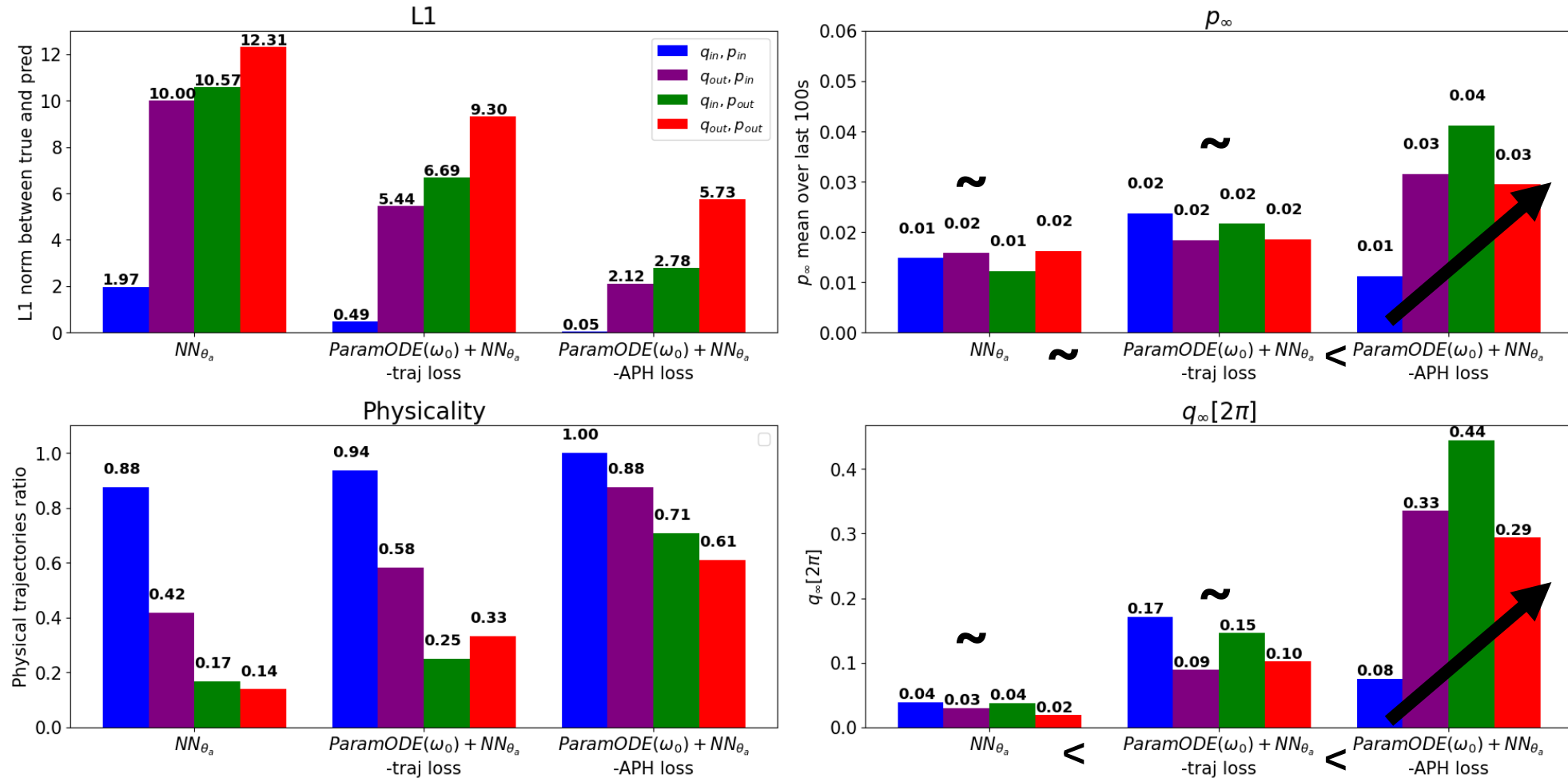


→ p out more impact than q out

→ NN alone is unable to produce physical trajectories outside the domain of training. The penalisation enhances the robustness of the hybrid model allowing 61% of physical trajectories vs only 33% without in

Q3.2 Generalization: Stationarity and robustness for out-of-distribution initial conditions

Comparison of metrics for different models and initial conditions



- NN alone better on metrics at infinity, but because many trajectories go back to $q_{\infty} = 0, 2\pi, -2\pi$ seen during training, which is not physical here
- The penalisation enhances physicality but introduces biases in final states



III. Numerical properties: error scheme

- **Order of time scheme (Norbert):**

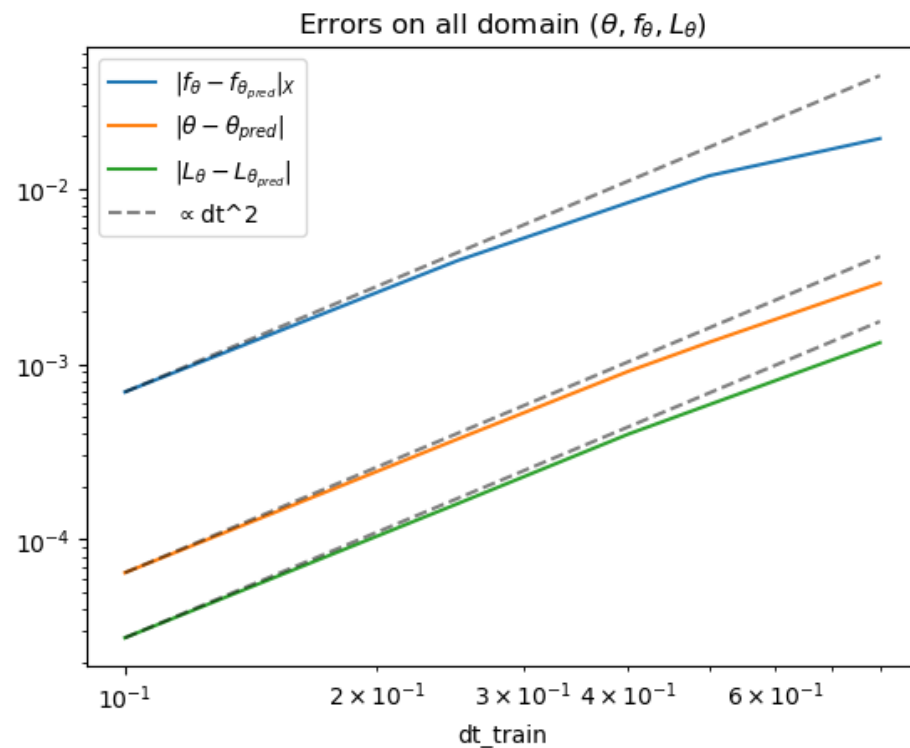
- Truth: ParamODE($\frac{2\pi}{12}, 0.2$)
- Train: ParamODE(ω_0, α)

RK4

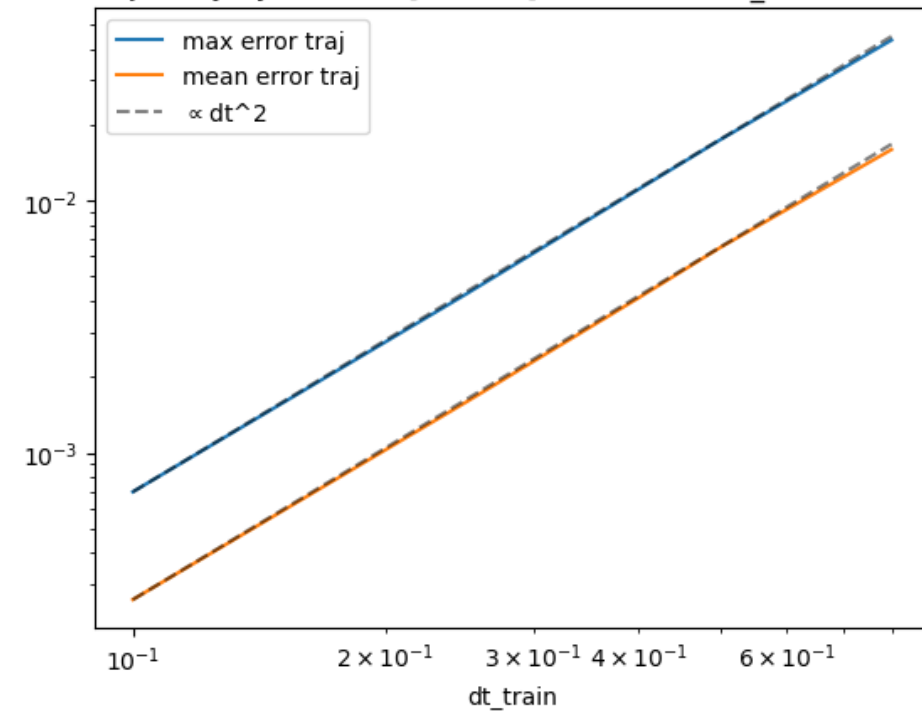
dt_truth=0.05

RK2

dt = [2,5,8,10,16]*dt_truth



Errors trajectory - y0 fixed = [0.5, 0.5], truth=RK4, dt_truth=0.05, t1=20





III. Estimating Lipschitz constants

- **Analytic Lipschitz constant**

$$F^{(\omega_0)}(q, p) = \begin{pmatrix} p(t) \\ -\omega_0^2 \sin(q(t)) \end{pmatrix} = \begin{pmatrix} F_1(q, p) \\ F_2(q, p) \end{pmatrix}$$

$$\nabla F^{(\omega_0)}(q, p) = \begin{pmatrix} \frac{\partial F_1}{\partial q} & \frac{\partial F_1}{\partial p} \\ \frac{\partial F_2}{\partial q} & \frac{\partial F_2}{\partial p} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -\omega_0^2 \cos(q) & 0 \end{pmatrix} \Rightarrow \|\nabla F\|_F = \sqrt{1 + \omega_0^4} = L$$

- **Estimated Lipschitz constant:**

Our models $F(q, p)$ are differentiable using **forward-mode auto-differentiation**.

We compute the Fröbenius norms of the jacobian for 100 pairs $(q, p) \in [-15, 15]^2$ and take the maximum one as our estimation of the Lipschitz constant.

```
jax.jacfwd(fun, argnums=0, has_aux=False, holomorphic=False)
```

Jacobian of `fun` evaluated column-by-column using forward-mode AD.

```
def model(y):  
    return jnp.array([y[1], -jnp.sin(y[0])])  
  
jax.jacfwd(model)(np.array([0., 0.]))  
✓ 0.3s  
  
Array([[ 0.,  1.],  
       [-1., -0.]], dtype=float32)
```

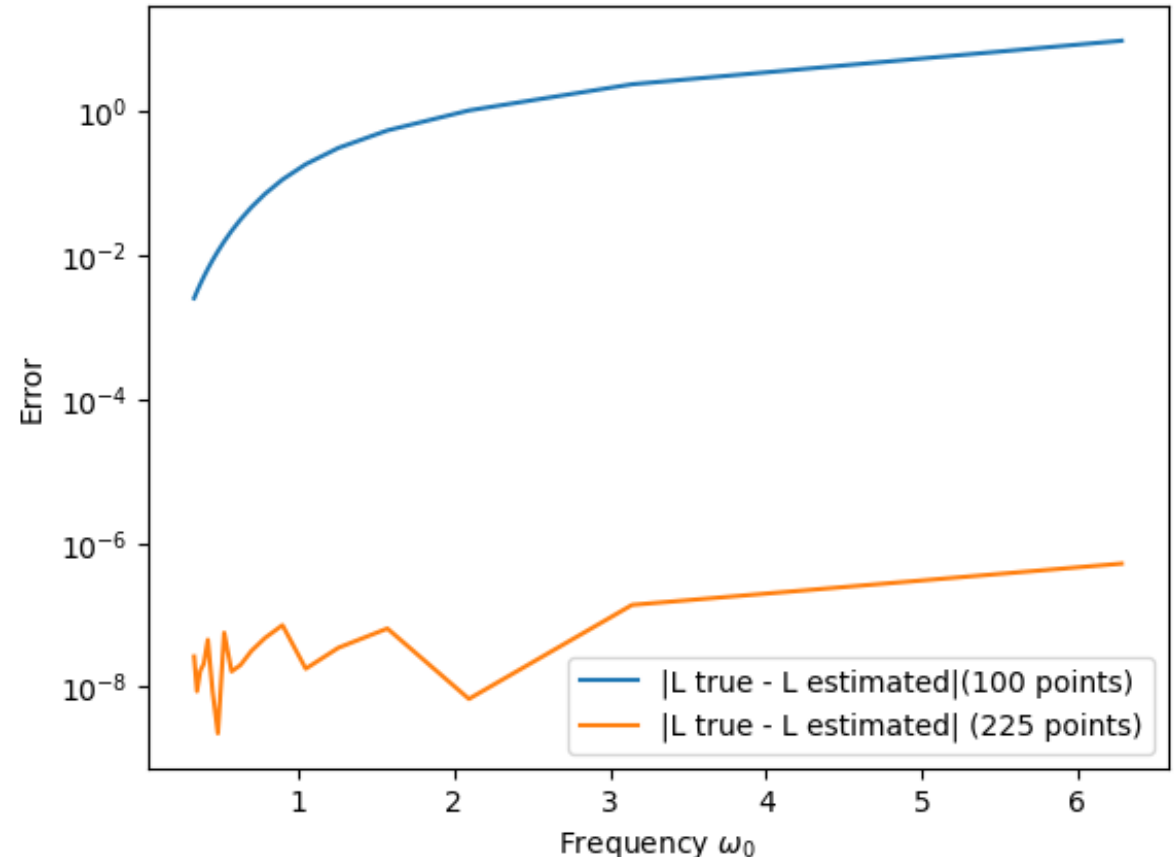


III. Estimating Lipschitz constants

- Error increases with frequency
→ increase number of points used in the estimation

→ We have a first method to estimate Lipschitz constants of neural networks, which we suspect play a role in the trajectory errors at inference, penalisation during training ?

True vs Estimated Lipschitz constant of the frictionless pendulum model



For NN, Fazlyab et al. (2019, NIPS), Després (2024)



III. A first upper bound on trajectory error

- After training, what type of error can we expect on the trajectory ?

$$\begin{cases} \dot{x}(t) = f(x(t)) \\ x(t=0) = x_0 \end{cases}$$

$$\begin{cases} \dot{y}(t) = f_\theta(y(t)) = f(y(t)) + \epsilon_\theta(y(t)) \\ y(t=0) = y_0 \end{cases}$$

L : Lipschitz constant of f

L_θ : Lipschitz constant of f_θ

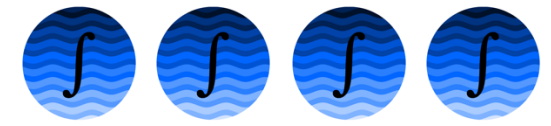
$$L_{\alpha,\theta} = \alpha L + (1 - \alpha)L_\theta$$

$$\epsilon_\theta = f_\theta - f$$

$$|x(t) - y(t)| \leq \exp(L_{\alpha,\theta}t) \left(|x_0 - y_0| + \int_0^t (\alpha |\epsilon_\theta(y(s))| + (1 - \alpha) |\epsilon_\theta(x(s))|) \exp(-L_{\alpha,\theta}s) ds \right)$$

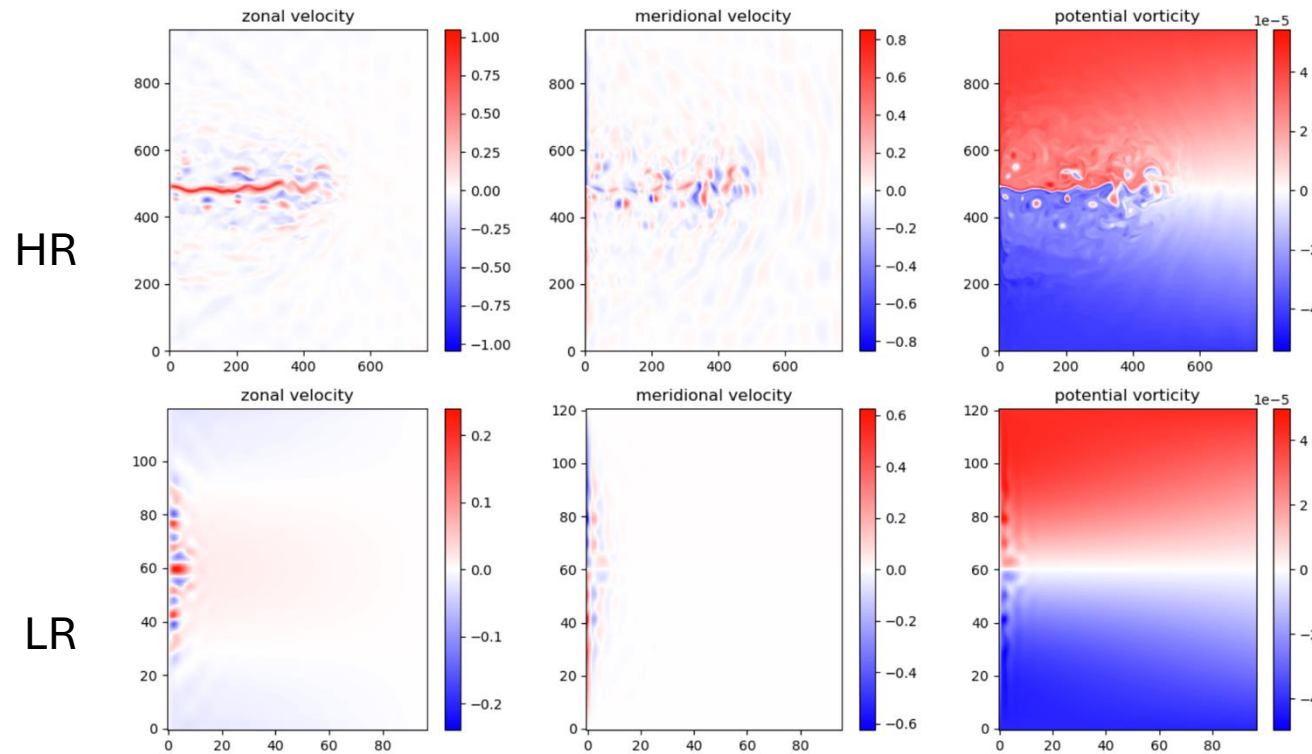
Special case $\alpha = 1$: if $0 \leq |\epsilon_\theta| \leq \epsilon_{max}$, we have $|x(t) - y(t)| \leq |x_0 - y_0| \exp(Lt) + \frac{|\epsilon_{max}|}{L} (\exp(Lt) - 1)$

Proof: Fundamental theorem of calculus, triangular inequality, f Lipschitz, Grönwall's inequality, IPP



IV. Towards ocean modelling

- QG: Recover Gulfstream with augmented dynamics, py-qg model from [Thiry, 2023](#)



[Thiry, 2023](#): constant correction

$$\partial_t q = \frac{1}{f_0} J(q, p) + f_0 B e + \frac{1}{f_0} (a_2 \Delta - a_4 \Delta^2) \Delta(p - p')$$

Us: dynamical correction

$$\partial_t q = \frac{1}{f_0} J(q, p) + f_0 B e + \frac{1}{f_0} (a_2 \Delta - a_4 \Delta^2) \Delta p + N N_\theta(q, p)$$

Work in progress...